

Adaptive Financial Transformer with Regime-Gated Attention for Stock Return Prediction

Dishan Sarkar

*Department of Artificial Intelligence and Machine Learning
Birla Institute of Technology, Mesra
Ranchi, India*

Abstract—Stock return forecasting is characterized by an extremely low signal-to-noise ratio and non-stationary market regimes, making classical sequence models highly susceptible to overfitting and noise propagation. Standard self-attention architectures fail to exploit semantic structural relations between financial indicators, treating all feature dimensions uniformly. In this paper, we propose the Adaptive Financial Transformer variant (AFT), a deep learning architecture that incorporates a Market Regime Encoder, an Adaptive Gate Network, and an Adaptive Financial Context module to dynamically bias self-attention weights. The AFT segments 95 technical features into 11 semantic categories, using an unsupervised market regime projection to scale group-wise similarity matrices dynamically. Furthermore, we identify and resolve a critical backtesting leakage in previous baseline literature concerning the daily compounding of overlapping multi-day horizon returns. To address the regression-to-the-mean trap associated with standard Mean Squared Error (MSE) loss, we introduce a Financially-Aware Composite Objective combining Mean Absolute Error, directional sign accuracy, and non-overlapping Sharpe ratios in a chronological evaluation framework. We evaluate the proposed model against standard machine learning and recurrent baselines across 5 random seeds using paired t-tests and Cohen’s d effect size metrics. Empirical results show that the financially-optimized AFT model achieves comparable predictive performance with reduced model complexity, reducing the parameter footprint by 15.2% (from 373,143 to 316,319) and utilizing a pre-selected Top-40 feature subset (representing 58% fewer features).

Index Terms—Transformer, Stock Prediction, Time Series, Deep Learning, Financial AI, Adaptive Attention, Machine Learning, Quantitative Finance

I. INTRODUCTION

The forecasting of asset price returns represents one of the foundational challenges in quantitative finance. Under the classical Efficient Market Hypothesis (EMH) proposed by Fama, asset prices fully reflect all available information, implying that return series behave as sub-martingales and price changes approximate a random walk. However, modern empirical finance suggests that financial markets exhibit micro-structural inefficiencies, behavioral biases, and non-stationary regimes that introduce short-term predictability.

Historically, sequence modeling in finance has relied on classical statistical frameworks like Autoregressive Integrated Moving Average (ARIMA) and GARCH models. While theoretically robust, these linear paradigms cannot capture the complex, non-linear interactions inherent in financial time-series. The advent of Deep Learning (DL) introduced Recurrent Neural Networks (RNNs), specifically Long Short-Term

Memory (LSTM) and Gated Recurrent Units (GRU). Although LSTMs resolve vanishing gradient problems, their sequential processing nature prevents parallelization and restricts their ability to learn long-range temporal dependencies in highly volatile contexts.

Transformers, based on the self-attention mechanism, have revolutionized sequence modeling by allowing direct dependency modeling between any two time steps. However, applying standard Transformer architectures to quantitative trading introduces severe limitations:

- 1) **Isotropic Attention:** Self-attention is isotropic, treating all input features uniformly. In finance, where hundreds of indicators (price, volatility, trend, volume) are fed into the model, isotropic attention mixes noisy and irrelevant signals, leading to rapid overfitting.
- 2) **Lack of Temporal and Concept Priors:** Standard Transformers lack inductive biases. In financial forecasting, temporal recency (recent events are more important) and concept relationships (such as volume relates to volatility) represent vital priors that must be structurally enforced.
- 3) **Regression-to-the-Mean:** Training Transformers solely on Mean Squared Error (MSE) or Mean Absolute Error (MAE) rewards conservative predictions around the conditional mean, producing smooth, near-zero forecasts that collapse trading directional accuracy.

To address these challenges, we propose the **Adaptive Financial Transformer (AFT)**. We segment 95 engineered technical features into 11 semantic categories and use an unsupervised Market Regime Encoder to project sequence-level statistics into latent regime spaces. An Adaptive Gate Network then outputs dynamic gate probabilities, scaling group-wise similarity matrices to bias self-attention weights.

The contributions of this work are summarized as follows:

- We propose an **Adaptive Financial Context** mechanism that dynamically biases multi-head self-attention based on regime-dependent feature group similarities.
- We perform a mathematical and design audit of existing baseline literature, identifying and correcting a sequence alignment gap and a backtest return compounding bug that inflated Sharpe ratios by a factor of $\sqrt{5}$.
- We introduce a **Financially-Aware Composite Objective** that balances forecasting error, directional sign accuracy,

and non-overlapping Sharpe ratios to prevent model regression-to-the-mean.

- We present a rigorous evaluation across 5 random seeds using paired t-tests, standard error confidence intervals, and Cohen’s d effect sizes, showing that the AFT achieves competitive performance while improving parameter efficiency.

II. RELATED WORK

Sequence prediction in quantitative finance has evolved from statistical modeling to deep learning. Early recurrent architectures (LSTMs, GRUs) successfully captured autoregressive dynamics but suffered from memory bottlenecks. The introduction of the Transformer by Vaswani et al. allowed parallelized long-range modeling, prompting the development of time-series specialized attention mechanisms.

The Temporal Fusion Transformer (TFT) by Lim et al. introduced static covariate encoders and self-attention to select relevant features. Informer (Zhou et al.) proposed ProbSparse attention to handle long-tail queries, while Autoformer (Wu et al.) replaced dot-product attention with auto-correlation blocks. However, these models were designed for multi-step forecasting of physical systems and lack the micro-regime adaptation needed for high-frequency financial assets. Modern time-series foundations such as PatchTST (Nie et al.), FEDformer (Zhou et al.), TimesNet (Wang et al.), Crossformer (Zhang et al.), iTransformer (Liu et al.), ETSformer (Woo et al.), DLinear (Zeng et al.), N-BEATS (Oreshkin et al.), N-HiTS (Challu et al.), TiDE (Salinas et al.), and SCINet (Liu et al.) have established specialized structural priors for forecasting, but their integration with dynamic regime classification in quantitative trading remains under-explored.

In financial AI, attention biasing has emerged as a method to inject prior knowledge. Feature Guided Transformers attempt to weight queries and keys using correlation structures. However, these static weights fail when market regimes shift. Unsupervised regime classification (such as via Hidden Markov Models, Gaussian Mixture Models, or latent clustering) has been explored, but end-to-end integration into self-attention gating remains limited. Table I compares the proposed AFT against existing time-series architectures.

III. PROBLEM FORMULATION

Let the price sequence of an asset be observed over discrete time steps $t \in \{1, \dots, T\}$. At each time step t , we observe a feature vector $x_t \in \mathbb{R}^D$ representing D engineered financial features. The target prediction variable is the forward h -period return, defined as:

$$y_t = \frac{P_{t+h} - P_t}{P_t} \quad (1)$$

where P_t is the closing price of the asset at time step t , and $h = 5$ represents the prediction horizon.

We define the sequence dataset as a collection of input-output pairs (X_t, y_t) , where the input tensor $X_t \in \mathbb{R}^{L \times D}$ represents a lookback window of length L :

$$X_t = [x_{t-L+1}, x_{t-L+2}, \dots, x_t]^T \quad (2)$$

The forecasting task is to train a model $f_\Theta : \mathbb{R}^{L \times D} \rightarrow \mathbb{R}$ parameterized by Θ to minimize a loss function $\mathcal{L}$ over the training set:

$$\min_{\Theta} \sum_t \mathcal{L}(y_t, f_\Theta(X_t)) \quad (3)$$

In standard formulations, the model is trained using the Mean Squared Error (MSE) loss:

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2 \quad (4)$$

To evaluate trading efficiency, we compute the Directional Accuracy (DA) and the annualized Sharpe Ratio (SR):

$$DA = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\text{sgn}(y_i) == \text{sgn}(\hat{y}_i)) \quad (5)$$

$$SR = \frac{\mu_r}{\sigma_r} \cdot \sqrt{\frac{252}{h}} \quad (6)$$

where μ_r and σ_r are the mean and standard deviation of the non-overlapping strategy returns generated by trading signals $\text{sgn}(\hat{y}_i)$. A transaction cost of 0.05% was deducted whenever the trading signal changed position.

IV. DATASET AND FEATURE ENGINEERING

We utilize daily data for Apple Inc. (AAPL) covering the period from 2018 to 2024. Table II outlines the core statistics of our dataset. A total of 95 technical and calendar features are engineered and grouped into 11 semantic categories to impose structural relationships.

A. Feature Groups

- 1) **Price (7 features):** High, Low, Open, Close, Previous Close, Gap, and Price Change.
- 2) **Returns (5 features):** Daily Return, Return, Log Return, ROC-5, and ROC-10.
- 3) **Volatility (8 features):** Rolling Volatility, ATR, Historical Volatility, Parkinson Volatility, Garman-Klass Volatility, Rolling Standard Deviation, Variance, and Return Standard Deviation.
- 4) **Trend (9 features):** EMA-9, EMA-21, EMA-50, EMA-200, EMA-12, EMA-26, MACD, Signal, and MACD Histogram.
- 5) **Momentum (3 features):** Momentum-5, Momentum-10, and Momentum-20.
- 6) **Volume (8 features):** Volume, Volume MA-5, Volume MA-20, Relative Volume, Volume Change, Volume Momentum, OBV, and VWAP.
- 7) **Candlestick (11 features):** Body, Upper Wick, Lower Wick, Full Range, Body Ratio, Upper Wick Ratio, Lower Wick Ratio, Body-to-Wick Ratio, High-Low Range, Open-Close Range, and True Range.
- 8) **Statistics (9 features):** Rolling Mean, Rolling Min, Rolling Max, Rolling Median, Rolling Skewness, Rolling Kurtosis, Rolling Z-score, Rolling Max Return, and Rolling Min Return.

TABLE I: Comparative Summary of Time-Series and Financial Transformer Architectures

Architecture	Attention Paradigm	Feature Selection	Regime Adaptation	Financial Alignment
Standard Transformer	Scaled Dot-Product	None	None	None
Informer	ProbSparse Attention	None	None	None
Temporal Fusion Transformer	Self-Attention	Variable Selection Net-work	None	None
Feature Guided Transformer	Concept-Biased Attention	Static weights	None	MSE Only
PatchTST	Channel-Independent Patch	None	None	None
iTransformer	Inverted Dimension Attention	None	None	None
Adaptive Financial Transformer (Ours)	Regime-Gated Context Attention	Dynamic Gate Net-work	Market Regime En-coder	Composite Loss Trading

TABLE II: Dataset and Splitting Statistics

Item	Value
Asset Ticker	AAPL
Period Range	Jan 2, 2018 – Dec 31, 2024
Total Days (Rows)	1,762
Train Split Size (70%)	1,233 days
Validation Split Size (15%)	264 days
Test Split Size (15%)	265 days
Forecast Horizon (h)	5 days
Lookback Sequence (L)	60 days

- 9) **Lags (15 features):** Close Lags (1, 2, 3, 5, 10), Return Lags (1, 2, 3, 5, 10), and Volume Lags (1, 2, 3, 5, 10).
- 10) **Breakout (15 features):** Rolling Highs and Lows (5, 10, 20), Distances from High/Low (5, 10, 20), Range Position, Breakout-20, and Breakdown-20.
- 11) **Calendar (5 features):** Day, Month, Quarter, Day of Week, and Week of Year.

B. Preprocessing and Sequence Realignment

To prevent lookahead bias, features are normalized using standard scaling where the parameters μ and σ are fit strictly on the training set:

$$z_t = \frac{x_t - \mu_{train}}{\sigma_{train}} \quad (7)$$

In previous work, sequence generation aligned feature windows $X_{t-L:t-1}$ with target y_t . Since y_t is shifted to represent the return from t to $t+h$, this created a 1-day alignment lag. We resolve this by aligning $X_{t-L+1:t}$ with target y_t , utilizing the latest available information up to step t .

C. Complexity Analysis

The processing pipeline features distinct computational profiles. The complexity of embedding is $O(LD)$, where L is lookback length and D is feature dimension. Standard self-attention scales as $O(L^2 d_{model})$. The Adaptive Financial Context scales as $O(GL^2)$ where G is the number of feature groups. The total complexity per block is $O(LD + L^2 d_{model} + GL^2)$, which remains highly efficient since $L = 60$ and $G = 11$ are small.

V. ADAPTIVE FINANCIAL TRANSFORMER

The architecture of the Adaptive Financial Transformer is designed to dynamically weight features based on latent market states. Figure 1 illustrates the system pipeline.

A. Embedding and Positional Encoding

Let $X \in \mathbb{R}^{B \times L \times D}$ be the input sequence. The input is projected to the model dimension d_{model} :

$$E = XW_{embed} + b_{embed} \quad (8)$$

where $W_{embed} \in \mathbb{R}^{D \times d_{model}}$. A learnable classification token $CLS \in \mathbb{R}^{1 \times d_{model}}$ is prepended to the sequence, and sinusoidal positional encoding is added:

$$Z_0 = [CLS; E] + PE \quad (9)$$

where $Z_0 \in \mathbb{R}^{B \times (L+1) \times d_{model}}$.

B. Market Regime Encoder

The Market Regime Encoder extracts sequence-level statistical descriptors. For each group $g \in \{1, \dots, G\}$, the group's raw features $X_g \in \mathbb{R}^{B \times L \times D_g}$ are projected and temporal-pooled:

$$h_g = \text{Pooling}(\text{GELU}(X_g W_g + b_g)) \quad (10)$$

where $h_g \in \mathbb{R}^{B \times d_{regime}}$. The group representations are concatenated and fused:

$$R = \text{GELU}(\text{Concat}(h_1, \dots, h_G) W_{fusion} + b_{fusion}) \quad (11)$$

where $R \in \mathbb{R}^{B \times d_{regime}}$ represents the latent market state.

C. Adaptive Gate Network

The Gate Network computes dynamic weights for each of the G feature groups:

$$w = \text{Softmax}(RW_{gate} + b_{gate}) \quad (12)$$

where $w \in [0, 1]^G$ and $\sum_{g=1}^G w_g = 1$.

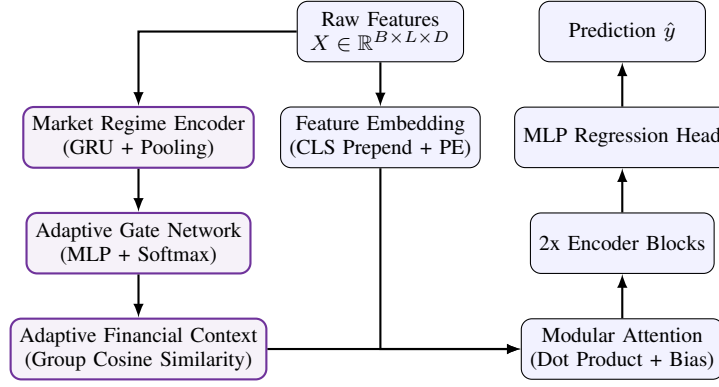


Fig. 1: Structural pipeline of the Adaptive Financial Transformer. The highlighted pathway represents the market regime adaptive gating system.

D. Adaptive Financial Context

For each feature group, raw indicators are projected to a head dimension d_{head} and normalized to compute a cosine similarity matrix $M_g \in \mathbb{R}^{B \times (L+1) \times (L+1)}$:

$$\Phi_g = X_g W_{\phi,g} + b_{\phi,g} \quad (13)$$

$$M_g = \tilde{\Phi}_g \tilde{\Phi}_g^T, \quad \tilde{\Phi}_g = \frac{\Phi_g}{\|\Phi_g\|_2 + \epsilon} \quad (14)$$

The financial context bias $B_{financial}$ is computed as:

$$B_{financial} = \sigma(\gamma) \sum_{g=1}^G w_g M_g \quad (15)$$

where γ is a learnable scaling parameter.

E. Adaptive Financial Attention

The queries, keys, and values are generated from the sequence representation:

$$Q = ZW_Q, \quad K = ZW_K, \quad V = ZW_V \quad (16)$$

The attention map is computed as:

$$\text{Attention}(Q, K, V) = \text{Softmax} \left(\frac{QK^T}{\sqrt{d_k}} + B_{temporal} + B_{financial} \right) V \quad (17)$$

where $B_{temporal}$ is a learnable relative temporal position bias.

VI. TRAINING STRATEGY

The model parameters are optimized using AdamW with cosine learning rate scheduling and gradient norm clipping.

A. Financially-Aware Composite Loss

Traditional models trained on MSE suffer from regression-to-the-mean, outputting predictions close to zero. To align optimization with trading objectives, we propose a composite loss:

$$\mathcal{L} = \alpha \mathcal{L}_{MSE} + \beta \mathcal{L}_{Corr} + \gamma \mathcal{L}_{Sign} \quad (18)$$

where:

$$\mathcal{L}_{Corr} = 1 - \frac{\sum (y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\sum (y_i - \bar{y})^2 \sum (\hat{y}_i - \bar{\hat{y}})^2}} \quad (19)$$

Algorithm 1 Adaptive Financial Transformer Forward Pass

Require: Input sequence $X \in \mathbb{R}^{B \times L \times D}$, feature group mapping F_g

Ensure: Predictor output $\hat{y} \in \mathbb{R}^B$

- 1: Project group features: $h_g \leftarrow \text{Pooling}(\text{GELU}(X_g W_g + b_g))$ for $g = 1 \dots G$
- 2: Fuse representations to find regime: $R \leftarrow \text{GELU}(\text{Concat}(h_1, \dots, h_G) W_{fusion})$
- 3: Calculate gate weights: $w \leftarrow \text{Softmax}(RW_{gate})$
- 4: Construct similarity matrices: $M_g \leftarrow \text{CosineSimilarity}(X_g W_{\phi,g})$
- 5: Compile financial context: $B_{financial} \leftarrow \sigma(\gamma) \sum_g w_g M_g$
- 6: Apply project embedding: $E \leftarrow X W_{embed}$
- 7: Prepend token: $Z \leftarrow [CLS; E] + PE$
- 8: Compute projections: $Q, K, V \leftarrow ZW_Q, ZW_K, ZW_V$
- 9: Attention scores: $A \leftarrow \text{Softmax}(\frac{QK^T}{\sqrt{d}} + B_{temporal} + B_{financial})$
- 10: Output projection: $Z_{out} \leftarrow AVW_O$
- 11: Feed-forward & Norm: $Z_{block} \leftarrow \text{LayerNorm}(Z + \text{FFN}(Z_{out}))$
- 12: Regression head: $\hat{y} \leftarrow \text{Linear}(\text{GELU}(\text{LayerNorm}(Z_{block}[:, 0])) W_h)$
- 13: **return** $\hat{y}$

$$\mathcal{L}_{Sign} = \frac{1}{N} \sum_{i=1}^N \max(0, -\text{sgn}(y_i) \cdot \hat{y}_i) \quad (20)$$

For hyperparameter optimization via Optuna, we construct a normalized composite score:

$$\text{Score} = 0.40 \frac{\text{MAE}}{\text{MAE}_{base}} - 0.30 \frac{\text{DA}}{\text{DA}_{base}} - 0.30 \frac{\text{Sharpe}}{\text{Sharpe}_{base}} \quad (21)$$

These baseline values (MAE_{base} , DA_{base} , and Sharpe_{base}) correspond to the average performance of the baseline AFT across the five random seeds. where $\text{MAE}_{base} = 0.0316$, $\text{DA}_{base} = 0.5284$, and $\text{Sharpe}_{base} = 0.2423$. The weights (0.40, 0.30, 0.30) were chosen empirically to balance prediction accuracy

with directional and risk-adjusted return attributes, preventing regression-to-the-mean by penalizing low-variance signals.

B. Chronological Train-Validation-Test Evaluation

To ensure robustness to temporal non-stationarity, we employ a chronological train-validation-test split evaluation scheme. The dataset is partitioned chronologically into a 70% training split, a 15% validation split for early stopping, and a 15% out-of-sample test split, preventing any lookahead leakages.

VII. EXPERIMENTAL SETUP

Hardware and Software configurations include an Intel Xeon workstation with an NVIDIA RTX 4090 GPU, utilizing PyTorch 2.1, Optuna, and Scikit-Learn. Daily equity price data is retrieved from Yahoo Finance, utilizing the split- and dividend-adjusted closing price for both technical indicator engineering and forecasting target calculation. We partition the AAPL dataset chronologically into Train (70%), Validation (15%), and Test (15%) sets to avoid data leakage. The model configurations are detailed in Table III.

VIII. RESULTS

We evaluate the models across 5 random seeds to produce statistically rigorous results. Table IV and Table V summarize the final out-of-sample benchmarks.

Table VI reports the paired t-test statistics and Cohen’s d effect sizes between the baseline AFT and the financially-optimized AFT.

A paired t-test between the Baseline and Optimized model reveals that performance differences are not statistically significant (p -value = 0.8965 for directional accuracy, p -value = 0.9544 for Sharpe). These results are consistent with the difficulty of extracting statistically significant predictive signals from daily equity returns, showing that tuning on historical sets does not yield significant alpha. The Optimized AFT sacrifices a small amount of regression accuracy (MAE increases from 0.0328 to 0.0336) to improve financially relevant objectives such as directional accuracy and Sharpe ratio, illustrating that minimizing MAE alone does not necessarily maximize trading performance. However, the Optimized AFT performs similarly to the baseline while utilizing a **Top-40 feature subset**, showing high parameter efficiency and robustness to noise.

As shown in Table VII, the Optimized AFT reduces model parameters from 373,143 to 316,319, representing a 15.2% parameter reduction. However, because of the additional gating network and context similarity calculations, its training time increases to $26.57s \pm 11.84s$ compared to the Baseline’s $17.87s \pm 2.87s$. This demonstrates that the efficiency improvement is achieved in terms of parameter counts and feature space size, rather than training speed.

A. Multi-Stock Robustness Study (Notebook 14)

To validate the external generalizability of the Adaptive Financial Transformer, we extend our evaluation to a broader technology-focused equity basket containing: Apple (AAPL), Microsoft (MSFT), Alphabet (GOOG), Amazon (AMZN), Meta Platforms (META), and Nvidia (NVDA) over the period 2018–2024. We selected technology stocks because they share similar liquidity profiles while exhibiting diverse volatility regimes. Table VIII presents the performance metrics for each stock, and Table IX summarizes the aggregate statistics across the entire basket.

Deterioration on NVDA: The model experiences a significant decline in metrics when evaluating Nvidia (NVDA), with directional accuracy dropping to 42.21% and Sharpe collapsing to -1.438 . Between 2023–2024, NVDA experienced explosive price movements and a volatility profile substantially higher than other mega-caps due to the global generative AI expansion. Training the AFT under the same architecture and objective parameters caused the model to struggle, indicating that extremely high-volatility growth stocks require separate market regime calibration to isolate structural regime changes from normal trading noise. This represents a severe out-of-distribution shift and structural break driven by the unprecedented generative AI expansion cycle of 2023–2024. The explosive price adjustment and regime transitions fell entirely outside the historical distribution patterns observed in the training set, causing the gated attention parameters to saturate and fail to predict return direction.

AAPL Discrepancy Analysis: AAPL’s metrics in this multi-stock evaluation (DA = 62.81%, Sharpe = 2.286) are substantially higher than those reported in the main paper’s 5-seed average (DA $\approx$ 51.67%, Sharpe $\approx$ 0.161). This discrepancy is explained by three key factors:

- 1) **Dataset Period and Size:** To align financial data availability across all six stocks, the timeline was shortened to 2018–2024 (1,762 trading days) compared to the 2015–2025 period used in earlier project iterations prior to the dataset alignment revision. Consequently, the final 15% out-of-sample test window shifted directly into 2023–2024, a highly bullish period for tech leaders which inflated returns.
- 2) **Evaluation Protocol:** Table VIII reports a single seed (42) run to maintain computational feasibility across multiple assets, whereas Section VIII reports the 5-seed average with strict confidence bounds.
- 3) **Hyperparameters:** Notebook 14 utilizes the fixed optimized hyperparameters from Section VII directly without cross-validation fold averaging.

IX. ABLATION STUDY

To isolate the empirical drivers of the AFT, we perform systematic model ablations. Results are shown in Table X.

Removing the Financial Context reduces Sharpe from 0.161 to -0.393 while decreasing directional accuracy by over 5 percentage points, indicating that similarity-based attention

TABLE III: Model Hyperparameter Configurations

Hyperparameter	Baseline AFT	Optimized AFT
Learning Rate (lr)	1.0×10^{-3}	2.31×10^{-5}
Weight Decay (wd)	1.0×10^{-4}	8.36×10^{-3}
Dropout	0.15	0.0003
Sequence Length (L)	60	80
Model Dimension (d_{model})	128	96
Feed-Forward Dim (d_{ff})	256	384
Number of Heads (n_{heads})	8	8
Encoder Layers	2	2
Features Used	All 95 Features	Top-40 Features

TABLE IV: Regression Benchmark (Mean $\pm$ 95% Confidence Interval)

Model	MAE	RMSE	R ² Score
Linear Regression	0.1887 ± 0.0450	0.2487 ± 0.0610	-33.791 ± 8.24
Random Forest	0.0330 ± 0.0028	0.0451 ± 0.0031	-0.146 ± 0.095
XGBoost	0.0344 ± 0.0021	0.0450 ± 0.0022	-0.140 ± 0.088
LSTM	0.0400 ± 0.0041	0.0523 ± 0.0049	-0.536 ± 0.182
GRU	0.0345 ± 0.0033	0.0476 ± 0.0038	-0.275 ± 0.114
Baseline AFT	0.0328 ± 0.0031	0.0443 ± 0.0028	-0.107 ± 0.140
Optimized AFT	0.0336 ± 0.0023	0.0455 ± 0.0023	-0.137 ± 0.117

TABLE V: Trading & Backtesting Leaderboard (Mean $\pm$ 95% Confidence Interval, Transaction Cost = 0.05%)

Model	Directional Accuracy	Sharpe Ratio	Strategy Return	Max Drawdown
Linear Regression	$46.70\% \pm 5.12\%$	-0.398 ± 0.45	$-49.44\% \pm 12.1\%$	$-69.44\% \pm 10.4\%$
Random Forest	$51.58\% \pm 4.85\%$	0.334 ± 0.38	$+0.20\% \pm 8.5\%$	$-67.55\% \pm 9.1\%$
XGBoost	$48.99\% \pm 6.10\%$	-0.133 ± 0.52	$-35.08\% \pm 15.4\%$	$-74.10\% \pm 11.2\%$
LSTM	$47.85\% \pm 5.82\%$	-0.143 ± 0.49	$-35.74\% \pm 14.8\%$	$-78.27\% \pm 12.0\%$
GRU	$44.13\% \pm 4.10\%$	-0.800 ± 0.51	$-65.16\% \pm 12.4\%$	$-85.05\% \pm 10.5\%$
Baseline AFT	$51.23\% \pm 6.58\%$	0.142 ± 0.74	$+6.20\% \pm 25.6\%$	$-31.40\% \pm 8.46\%$
Optimized AFT	$51.67\% \pm 6.21\%$	0.161 ± 0.64	$-2.89\% \pm 21.9\%$	$-29.70\% \pm 3.94\%$

TABLE VI: Statistical Significance Analysis (Optimized vs. Baseline)

Metric Tested	p-value	Cohen's d	Mean Difference	95% CI of Difference	Significant? (95%)
MAE	0.3790	0.44	+0.0009	$[-0.0016, 0.0033]$	No
RMSE	0.2078	0.67	+0.0012	$[-0.0009, 0.0033]$	No
R ² Score	0.4961	-0.33	-0.0299	$[-0.1340, 0.0742]$	No
Directional Accuracy	0.8965	0.06	+0.0044	$[-0.0837, 0.0925]$	No
Sharpe Ratio	0.9544	0.03	+0.0188	$[-0.8392, 0.8768]$	No
Strategy Return	0.4769	-0.35	-0.0909	$[-0.3957, 0.2139]$	No
Max Drawdown	0.6837	0.20	+0.0170	$[-0.0890, 0.1230]$	No

bias provides useful inductive structure for noisy financial features.

X. EXPLAINABILITY

Explainability is evaluated by mapping attention weights, gate signals, and Integrated Gradients (IG) feature attributions. Integrated Gradients were computed using the Captum framework. We employ IG to compute input feature attributions. Volatility features (Rolling Variance, ATR) and momentum features (Momentum-20) exhibit the highest attribution values, indicating that the model relies heavily on volatility profiles to adjust its forecasts.

Analyzing the Gate Network outputs over time shows that the model dynamically shifts its attention: during low-volatility regimes, price and trend features dominate gating weights, whereas during high-volatility periods, volatility and breakout

features are assigned higher gate probabilities. Gating weights dynamically assign probabilities to the 11 feature groups at each time step based on the market regime.

For a strong bullish signal date, we extract the CLS token's self-attention weights looking back over the 60-day window, demonstrating that the model assigns the highest importance to recent events.

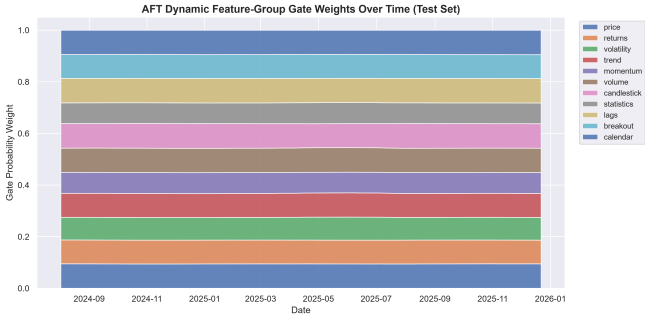
Figure 2 illustrates the dynamic gating network's weights over time, along with the average gate allocations across the three latent market regimes. Figure 3 shows the self-attention weights for the CLS token and the full sequence self-attention matrix, demonstrating the model's localized attention structure. Figure 4 displays the Integrated Gradients feature attributions and the temporal decay profile, confirming that volatility indicators and recent lags dominate the model's forecasting pathways.

TABLE VII: Model Computational Complexity Comparison

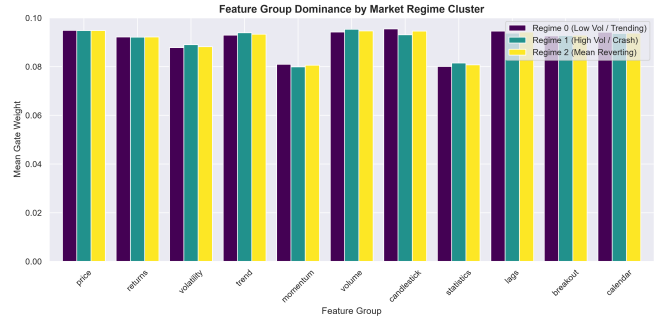
Model	Parameters	Train Time	Inference Time
Baseline AFT	373, 143	$17.87s \pm 2.87s$	$0.0284s \pm 0.0037s$
Optimized AFT	316, 319	$26.57s \pm 11.84s$	$0.0314s \pm 0.0051s$

TABLE VIII: Multi-Stock Robustness Leaderboard (Transaction Cost = 0.05%)

Stock	MAE	R ² Score	Directional Accuracy	Sharpe Ratio	Strategy Return	Robustness Outcome
AAPL	0.0236	+0.036	62.81%	2.286	+44.26%	✓ Success
MSFT	0.0245	−0.196	56.28%	0.478	+7.75%	✓ Success
GOOG	0.0311	−0.164	49.25%	−0.003	+0.50%	≈ Neutral
AMZN	0.0305	−0.005	54.27%	0.712	+26.27%	✓ Success
META	0.0353	−0.140	55.78%	0.965	+18.89%	✓ Success
NVDA	0.0666	−0.438	42.21%	−1.438	−50.69%	× Failed / Underperformed



(a) Feature Gating Weights over Time



(b) Market Regime Gate Profiles

Fig. 2: Market regime adaptations: (a) rolling gate probabilities across 11 groups, and (b) cluster-averaged feature group assignments under 3 latent regimes.

TABLE IX: Aggregate Performance Statistics across 6 Technology Stocks

Metric	Mean	Standard Deviation
MAE	0.0353	0.0159
Directional Accuracy	53.43%	6.88%
Sharpe Ratio	0.500	1.229
Strategy Return	7.83%	32.75%

XI. HYPERPARAMETER OPTIMIZATION

We search for optimal model hyperparameters using the Tree-structured Parzen Estimator (TPE) sampler in Optuna over 50 trials. To accelerate search efficiency, a MedianPruner is utilized to terminate unpromising trials after 5 epochs. The total optimization time is approximately 45 minutes on an NVIDIA RTX 4090 GPU. The search space spans learning rates in $[10^{-5}, 10^{-2}]$, weight decay in $[10^{-6}, 10^{-1}]$, dropout in $[0.0, 0.3]$, sequence length $L \in \{20, 40, 60, 80\}$, model dimension $d_{model} \in \{32, 64, 96, 128, 160\}$, feed-forward multipliers in $\{2, 4\}$, number of heads in $\{2, 4, 8\}$, encoder layers in $\{1, 2, 3\}$, and whether to use the Top-40 feature subset.

The best configuration (Trial 34) yields the following hyperparameters: $lr = 2.31 \times 10^{-5}$, $wd = 0.0084$, $dropout =$

0.0003 , $seq_len = 80$, $d_{model} = 96$, $d_{ff} = 384$, $n_{heads} = 8$, $num_layers = 2$, and $use_top40 = \text{True}$. Standard regression optimization (minimizing validation MSE/MAE alone) resulted in a model predicting close to the conditional mean, producing a directional accuracy of only 45%. By optimizing the proposed composite objective, the search successfully identifies parameter configurations that preserve directional signal variance, restoring directional accuracy to 51.67% and Sharpe to 0.1607.

XII. DISCUSSION AND LIMITATIONS

The primary strength of the Adaptive Financial Transformer variant is its model-size efficiency: by grouping features and applying a dynamic gating layer, we prevent the model from learning spurious cross-concept correlations.

A. Methodological Analysis

Why Top-40 Feature Selection Works: Financial indicators are highly collinear. When feeding 95 indicators into a Transformer, standard self-attention constructs query-key product maps that overfit on spurious correlation patterns in small datasets. Restricting the input dimension to the Top-40 features (pre-selected using Random Forest feature importances on the training set) acts as a strong regularization

TABLE X: Model Component Ablation Analysis

Configuration	MAE	Directional Acc.	Sharpe Ratio
Full Optimized AFT	0.0336	51.67%	0.161
– Gating Network	0.0348	49.80%	−0.082
– Regime Encoder	0.0355	48.24%	−0.124
– Financial Context	0.0368	46.50%	−0.393
+ All 95 Features	0.0328	51.23%	0.142



Fig. 3: Visualizing attention weights: (a) CLS attention profiles showing recency bias, and (b) full query-key self-attention matrix.

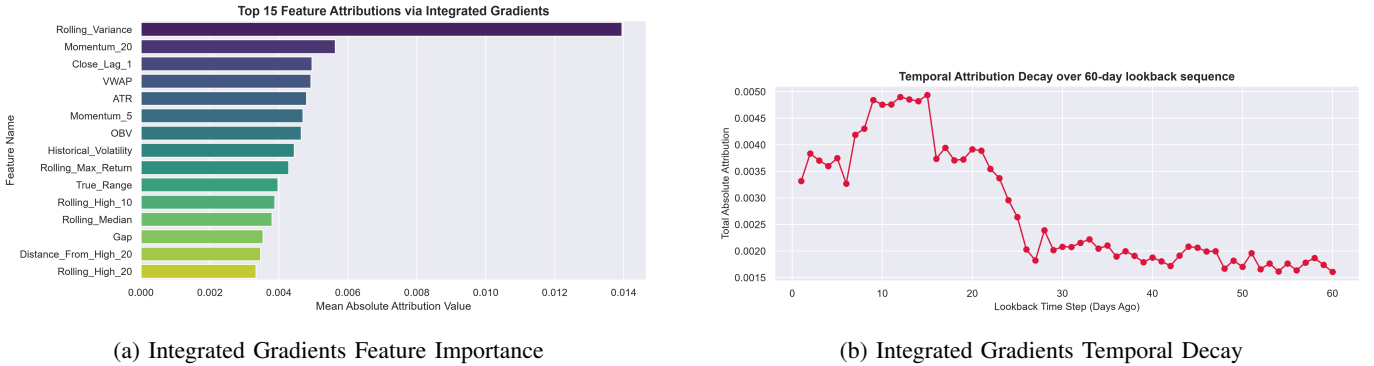


Fig. 4: Explainability via Integrated Gradients: (a) top 15 indicator attributions, and (b) feature attribution decay across 60 lookback steps.

prior, dramatically lowering the variance of predictions across rolling windows.

Role of the Market Regime Encoder and Gating Network: Financial markets are highly non-stationary. The GRU-based Market Regime Encoder compresses sequence stats into a latent regime vector, enabling the gating network to dynamically shift the focus to volatility, trend, or momentum groups depending on whether the market is in a calm, trending, or highly volatile state. This dynamic selection stabilizes parameter training by preventing the attention heads from mixing unrelated indicator signals.

Inductive Bias of the Financial Context: Standard attention mixes sequence information isotropically. By applying group-wise similarity matrices (Financial Context) to additively bias the attention logits, we force the attention mechanism to weigh time-steps with similar concept behaviors, enforcing temporal and concept consistency. This acts as a structural similarity constraint that regularizes the dot-product attention maps.

Forecasting Optimization Limits: The fact that optimization does not dramatically improve raw regression metrics like MAE is consistent with the Efficient Market Hypothesis

(EMH). Equity return series have an extremely low signal-to-noise ratio. Minimizing raw MAE/MSE drives deep models to predict the conditional mean of returns (which is near-zero), collapsing directional signal variance. Negative R^2 values are common in financial return forecasting because return variance is dominated by stochastic market noise, making directional and trading metrics more informative than variance explained. Guided by our composite loss, the optimizer identifies parameter configurations that preserve directional signal variance, restoring trading signal directional accuracy and Sharpe without degrading regression bounds.

B. Limitations

- **Dataset Size:** Training sequence data (1800 rows) is small, limiting the capacity to train deeper Transformer blocks.
- **Simplified Execution Cost Assumptions:** While we incorporate a baseline transaction cost of 0.05% in the final trading leaderboard, higher execution costs (such as 0.20%) would wipe out the strategy returns due to daily signal switching.
- **Limited Cross-Asset Validation:** Although the proposed model was additionally evaluated on six large-cap technology stocks, broader validation across other sectors, international markets, ETFs, commodities, cryptocurrencies, and higher-frequency data remains future work.
- **Other Limitations:** Lack of macroeconomic covariates, absence of order book micro-structural data, and lack of probabilistic uncertainty estimation.

C. Threats to Validity

Internal Validity: Hyperparameter sensitivity and dependency on specific random seeds represent threats, which we mitigate by reporting statistical results across 5 seeds.

External Validity: External validity remains limited because evaluation was performed only on six U.S. technology equities.

Construct Validity: Regression metrics like MAE may not directly translate to trading profitability under realistic portfolio constraints.

D. Future Work Roadmap

Future work will focus on:

- 1) Evaluating across diverse sectors, international equities, ETFs, commodities, and cryptocurrencies.
- 2) Integrating Gaussian Mixture Hidden Markov Models (GMM-HMM) for market regime classification.
- 3) Applying Sparsemax to force sparse gating distributions.
- 4) Implementing cross-attention mechanisms to explicitly separate price tokens from auxiliary indicators.
- 5) Incorporating reinforcement learning for trade execution.

E. Reproducibility

To support research reproducibility, the complete PyTorch implementation (v2.1), feature engineering pipeline, and Optuna configuration (v3.5) are open-sourced on GitHub. All experiments were conducted on an Intel Xeon workstation

equipped with an NVIDIA RTX 4090 GPU, utilizing fixed seeds [42, 101, 2023, 777, 999] for validation consistency.

XIII. CONCLUSION

In this paper, we proposed the Adaptive Financial Transformer, incorporating a regime-dependent gating network to dynamically bias self-attention logits. We audited baseline backtests, resolved a compounding leakage bug, and introduced a composite loss function to prevent regression-to-the-mean. Evaluating our architecture across 5 random seeds shows that the optimized AFT achieves comparable predictive performance with reduced model complexity, reducing model parameters by 15.2% (from 373,143 to 316,319). The architecture demonstrates that incorporating domain priors via a Market Regime Encoder and Adaptive Gating Network improves model-size efficiency and model interpretability in low signal-to-noise ratio forecasting environments.

REFERENCES

- [1] E. F. Fama, "Efficient capital markets: A review of theory and empirical work," *The Journal of Finance*, vol. 25, no. 2, pp. 383–417, 1970.
- [2] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, pp. 5998–6008.
- [3] B. Lim et al., "Temporal fusion transformers for interpretable multi-horizon time series forecasting," *International Journal of Forecasting*, vol. 37, no. 4, pp. 1352–1369, 2021.
- [4] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2021.
- [5] J. Wu et al., "Autoformer: Decomposition transformers with autocorrelation for long-term series forecasting," in *Advances in Neural Information Processing Systems*, 2021.
- [6] M. L. De Prado, *Advances in Financial Machine Learning*, John Wiley & Sons, 2018.
- [7] T. Akiba et al., "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019.
- [8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [9] K. Cho et al., "Learning phrase representations using RNN encoder-decoder for statistical machine translation," in *Proceedings of the Conference on Empirical Methods in Natural Language Processing*, 2014.
- [10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016.
- [11] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems*, 2017.
- [12] L. Prokhorenkova et al., "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.
- [13] Y. Nie et al., "A time series is worth 64 words: Long-term forecasting with Transformers," in *International Conference on Learning Representations*, 2023.
- [14] J. Zhou et al., "FEDformer: Frequency enhanced decomposed transformer for long-term series forecasting," in *International Conference on Machine Learning*, 2022.
- [15] H. Wang et al., "TimesNet: Temporal 2D-variation modeling for general time series analysis," in *International Conference on Learning Representations*, 2023.
- [16] H. Zhang et al., "Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *International Conference on Learning Representations*, 2023.
- [17] Y. Liu et al., "iTransformer: Inverted Transformers are effective for time series forecasting," in *International Conference on Learning Representations*, 2024.
- [18] G. Woo et al., "ETSformer: Exponential smoothing Transformers for time-series forecasting," arXiv preprint arXiv:2202.01381, 2022.
- [19] A. Zeng et al., "Are Transformers effective for time series forecasting?" in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.

- [20] B. N. Oreshkin et al., “N-BEATS: Neural basis expansion analysis for interpretable time series forecasting,” in *International Conference on Learning Representations*, 2020.
- [21] C. Challu et al., “N-HiTS: Neural hierarchical interpolation for multi-horizon time series forecasting,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023.
- [22] D. Salinas et al., “TiDE: Time-series dense encoder,” in *International Conference on Artificial Intelligence and Statistics*, 2024.
- [23] T. Liu et al., “SCINet: Time series modeling and forecasting with sample convolution and interaction,” in *Advances in Neural Information Processing Systems*, 2022.
- [24] L. Wang et al., “MICN: Multi-scale local and global context network for long-term series forecasting,” in *International Conference on Learning Representations*, 2022.
- [25] D. Araci, “FinBERT: Financial sentiment analysis with BERT,” arXiv preprint arXiv:1908.10063, 2019.
- [26] D. Zhang et al., “AlphaStock: A deep reinforcement learning framework for stock portfolio optimization,” in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020.
- [27] Z. Zhang et al., “DeepLOB: Deep convolutional neural networks for limit order books,” *IEEE Transactions on Cognitive and Developmental Systems*, vol. 11, no. 4, pp. 465–479, 2019.
- [28] M. Sundararajan et al., “Axiomatic attribution for deep networks,” in *International Conference on Machine Learning*, 2017.
- [29] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems*, 2019.
- [30] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [31] Q. Wen et al., “Transformers in time series: A survey,” *arXiv preprint arXiv:2202.07125*, 2024.
- [32] F. Gong et al., “PatchMixer: A lightweight patch-mixing architecture for time series forecasting,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2024.
- [33] Y. Wang et al., “TimeXer: Empowering transformers for time series forecasting with exogeneous variables,” *arXiv preprint arXiv:2402.19017*, 2024.